ALGORITMOS SIMPLES DE ORDENAÇÃO

DCE792 - AEDs II (Prática)

Atualizado em: 17 de outubro de 2026



Iago Carvalho

Departamento de Ciência da Computação

ORDENAÇÃO

Ordenação é uma das tarefas mais básicas existentes tem computação

- Colocar uma série de itens em ordem
  - Crescente
  - Decrescente
- Podemos trabalhar com qualquer tipo de item
  - Inteiros
  - Palavras
  - Datas
  - ...

Um algoritmo de ordenação é aquele que atua sobre um conjunto de elementos e os coloca em ordem

- Na maioria das vezes, este algoritmo atua sobre um vetor

ORDENAÇÃO

A ordenação é baseada em uma chave

- A chave de ordenação é o **campo** utilizado para comparação
  - Valor de uma variável
  - Campo *nome* em uma struct
  - Determinada coluna em uma tabela
  - ...

Comparando uma chave com outra é que sabemos se um determinado elemento está ou não a frente de outros no conjunto

CHAVE DE ORDENAÇÃO

Após a escolha da chave, devemos também definir a regra de ordenação

- Como dizer se uma chave é menor, igual ou maior que outra

Podemos utilizar ordem numérica, alfabética, alfa-numérica

A ordenação pode ser crescente ou decrescente de acordo com a chave e regra escolhida

Também podem existir regras mais complexas de ordenação

- Regras multi-chaves lexicográficas

ORDENAÇÃO INTERNA OU EXTERNA

Temos duas grandes classes de algoritmos de ordenação

Ordenação interna: assunto desta disciplina

- Conjunto de dados cabe na RAM
- Qualquer elemento pode ser imediatamente acessado

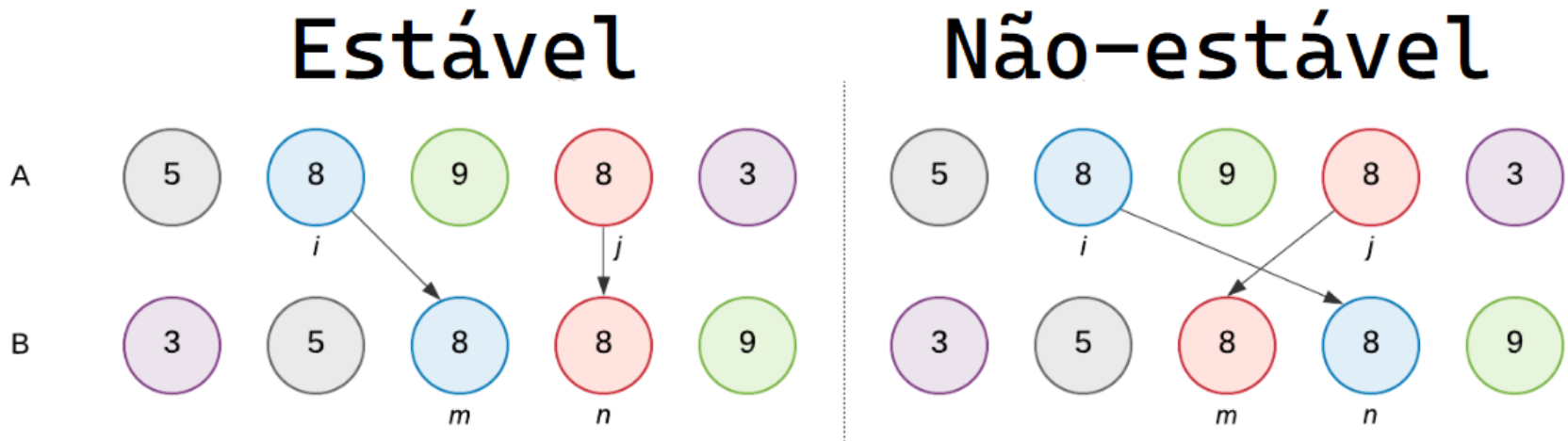
Ordenação externa

- Conjunto de dados não cabe na RAM
  - Devem ser armazenados em memória secundária
- Dados podem ser acessados em ordem sequencial ou em grandes blocos

ORDENAÇÃO ESTÁVEL OU NÃO ESTÁVEL

Um algoritmo é dito ser **estável** caso a ordem dos elementos com chaves iguais não seja mudada durante a ordenação

- Caso a ordem possa ser mudada, o algoritmo é considerado **não-estável**



MÉTODOS DE ORDENAÇÃO

Nesta aula, estudaremos métodos de ordenação simples

- Tempo quadrático
- Fácil implementação
- Auxiliam o entendimento de algoritmos mais complexos

Vamos estudar 3 algoritmos

- Bubble sort
- Selection sort
- Insertion sort

BUBBLE SORT

BUBBLE SORT

Método extremamente simples (e pouco eficiente) de ordenação

Compara pares de valores adjacentes e os troca de lugar caso necessário

- Trabalha fazendo comparações simples
- Compara todos os pares de elementos

O processo é realizado de forma iterativa até que o vetor inteiro esteja ordenado

BUBBLE SORT – PSEUDOCÓDIGO

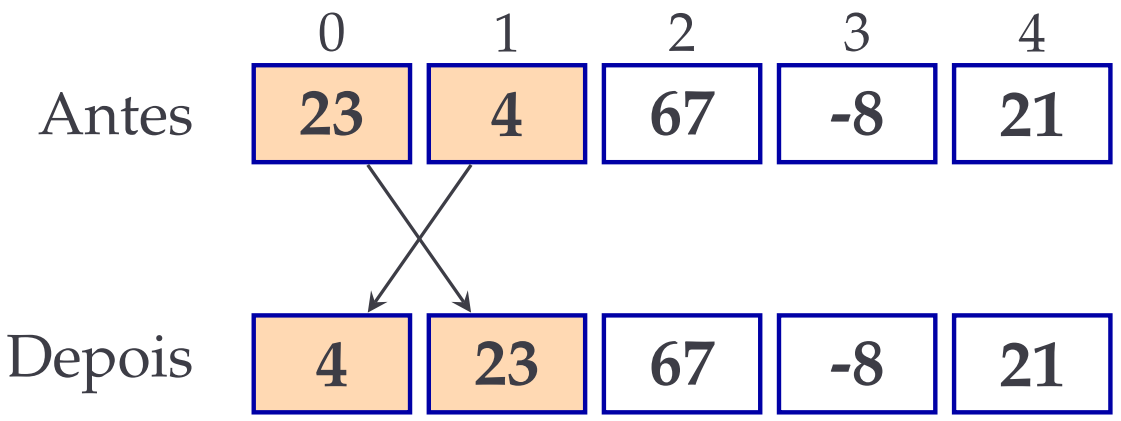
Algoritmo 1: Bubble sort

Entrada: Vetor $A[0 \ldots n-1]$ com n elementos
Saída : Vetor A ordenado

```
1 n ← tamanho(A);
2 para cada i ∈ {0, ..., n − 2} faça
3     para cada j ∈ {0, ..., n − i − 2} faça
4         se A[j] > A[j + 1] então
5             troque A[j] e A[j + 1];
```

BUBBLE SORT

Primeira comparação: $i = 0$



$23 > 4$, então os valores adjacentes são trocados.

BUBBLE SORT

1ª passagem: o maior valor chega ao fim

$i = 0$	**23**	**4**	**67**	**-8**	**21**	23 > 4: troca
$i = 1$	**4**	**23**	**67**	**-8**	**21**	23 < 67: mantém
$i = 2$	**4**	**23**	**67**	**-8**	**21**	67 > −8: troca
$i = 3$	**4**	**23**	**-8**	**67**	**21**	67 > 21: troca
Final	**4**	**23**	**-8**	**21**	**67**	Maior valor fixado no fim.

BUBBLE SORT

2ª passagem

$i = 0$	**4**	**23**	**-8**	**21**	**67**	$4 < 23$: mantém
$i = 1$	**4**	**23**	**-8**	**21**	**67**	$23 > -8$: troca
$i = 2$	**4**	**-8**	**23**	**21**	**67**	$23 > 21$: troca
Final	**4**	**-8**	**21**	**23**	**67**	Dois maiores valores fixados.

BUBBLE SORT

3ª e 4ª passagens

$i = 0$	**4**	**-8**	**21**	**23**	**67**	$4 > -8$: troca
$i = 1$	**-8**	**4**	**21**	**23**	**67**	$4 < 21$: mantém
Final 3ª	**-8**	**4**	**21**	**23**	**67**	Fim da 3ª passagem.
$i = 0$	**-8**	**4**	**21**	**23**	**67**	$-8 < 4$: mantém; sem trocas.
Ordenado	**-8**	**4**	**21**	**23**	**67**	Vetor ordenado.

SELECTION SORT

SELECTION SORT

Outro método simples e pouco eficiente de ordenação

- Também conhecido como ordenação por seleção

Método iterativo

- A cada iteração, seleciona o item com melhor ordem do vetor
  - De acordo com a regra de ordenação
- Insere o item selecionado na primeira posição
- Destaca-se a primeira posição do vetor e repete o processo
  - Processo é repetido até que todas as posições sejam destacadas

Na prática, é um pouco superior ao Bubble Sort

- Faz um número menor de comparações

SELECTION SORT – PSEUDOCÓDIGO

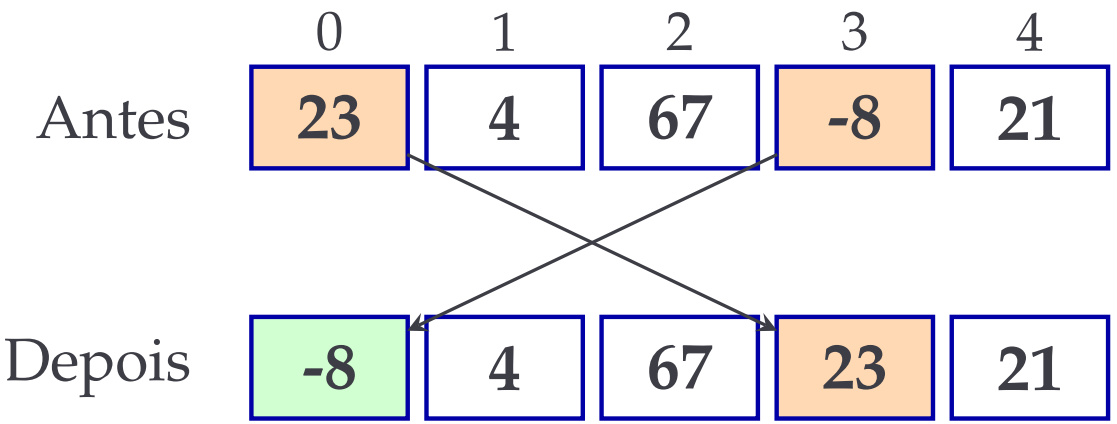
Algoritmo 2: Selection sort

Entrada: Vetor $A[0 \ldots n-1]$ com n elementos
Saída : Vetor A ordenado

```
1 n ← tamanho(A);
2 para cada i ∈ {0, ..., n − 2} faça
3     menor ← i;
4     para cada j ∈ {i + 1, ..., n − 1} faça
5         se A[j] < A[menor] então
6             menor ← j;
7     se menor ≠ i então
8         troque A[i] e A[menor];
```

SELECTION SORT – EXEMPLO

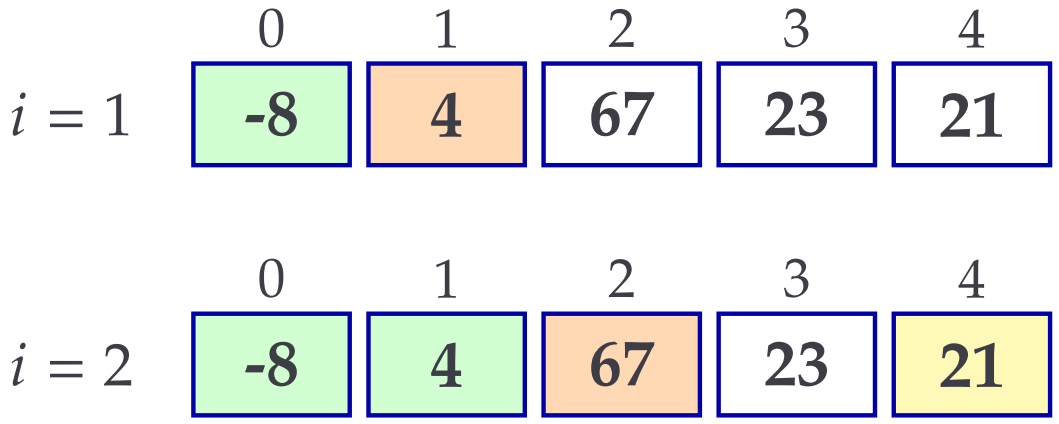
Primeira passagem: $i = 0$



O menor valor da lista é -8. Troca $V[0]$ com $V[3]$.

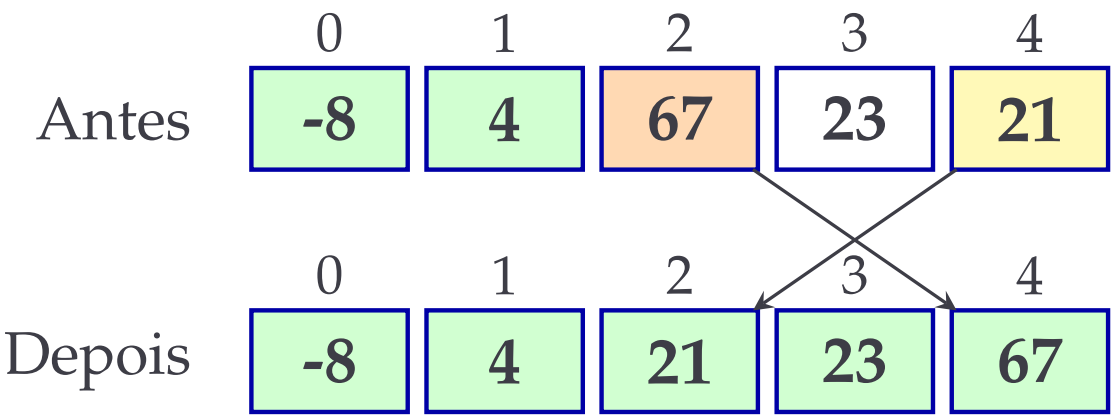
SELECTION SORT – EXEMPLO

Próximas passagens



SELECTION SORT – EXEMPLO

Troca e término



$i = 2$: troca 67 com 21.
$i = 3$: já está na posição correta.

Vetor ordenado

INSERTION SORT

INSERTION SORT

Um terceiro método simples e pouco eficiente de ordenação

- Também conhecido como ordenação por inserção

Similar ao processo que você realiza ao ordenar as cartas de baralho em sua mão

- Pegue uma carta por vez e a insira na posição correta
- Processo realizado de forma iterativa
  - Até que toda sua mão esteja ordenada



INSERTION SORT - FUNCIONAMENTO

O algoritmo percorre o vetor inteiro

- Para cada posição, verifica se a chave está corretamente posicionada
  - Isto é realizado percorrendo o vetor na ordem inversa
  - A partir da posição analisada até o início do vetor
- Caso a chave esteja corretamente posicionada, não realiza nenhuma ação
- Caso contrário, a insere na posição correta
  - Necessário deslocar todas as outras chaves

Esse processo de deslocamento é barato caso seja utilizado uma lista encadeada

- Necessário somente uma operação com ponteiros

Caso seja um vetor estático, é necessário literalmente trocar todas as posições de memória

INSERTION SORT – PSEUDOCÓDIGO

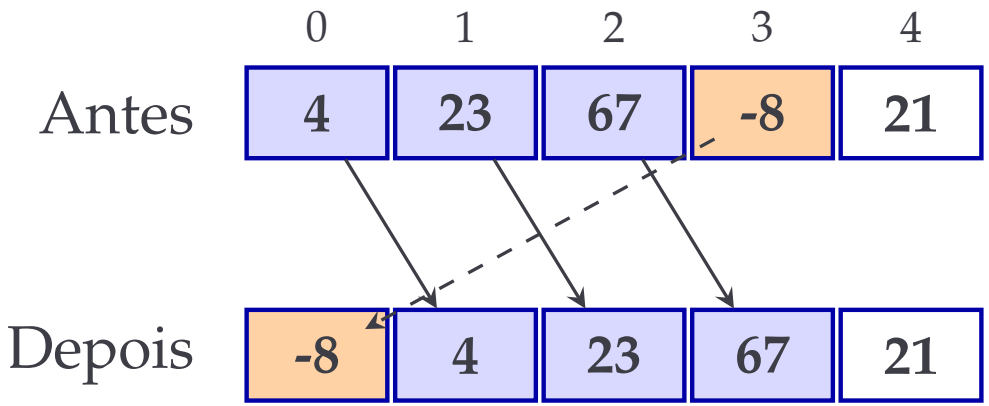
Algoritmo 3: Insertion sort

Entrada: Vetor $A[0 \ldots n-1]$ com n elementos
Saída : Vetor A ordenado

```
1 n ← tamanho(A);
2 para cada i ∈ {1, ..., n − 1} faça
3     chave ← A[i];
4     j ← i − 1;
5     enquanto j ≥ 0 e A[j] > chave faça
6         A[j + 1] ← A[j];
7         j ← j − 1;
8     A[j + 1] ← chave;
```

INSERTION SORT

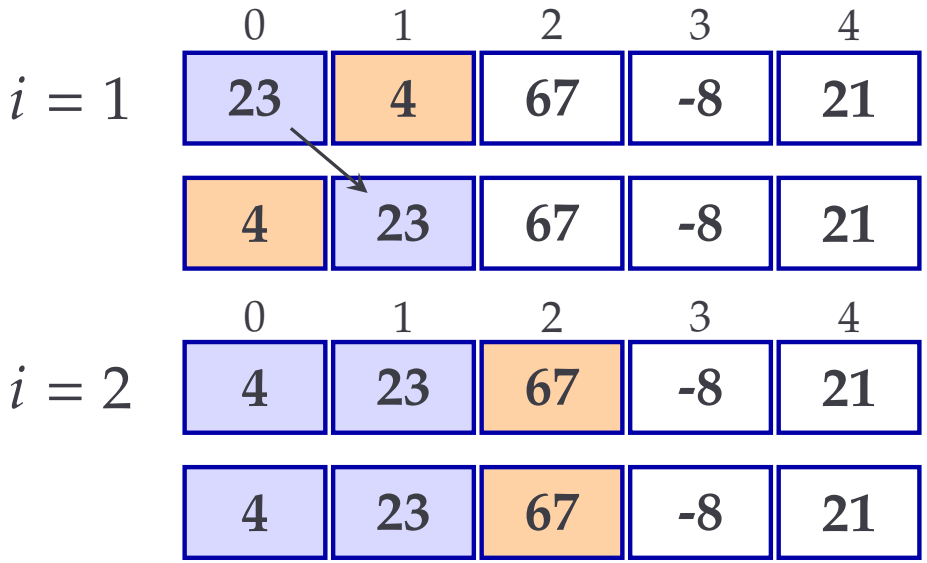
Inserção da chave na parte ordenada



Desloca as chaves maiores para a direita e insere −8 na posição que ficou vaga.

INSERTION SORT

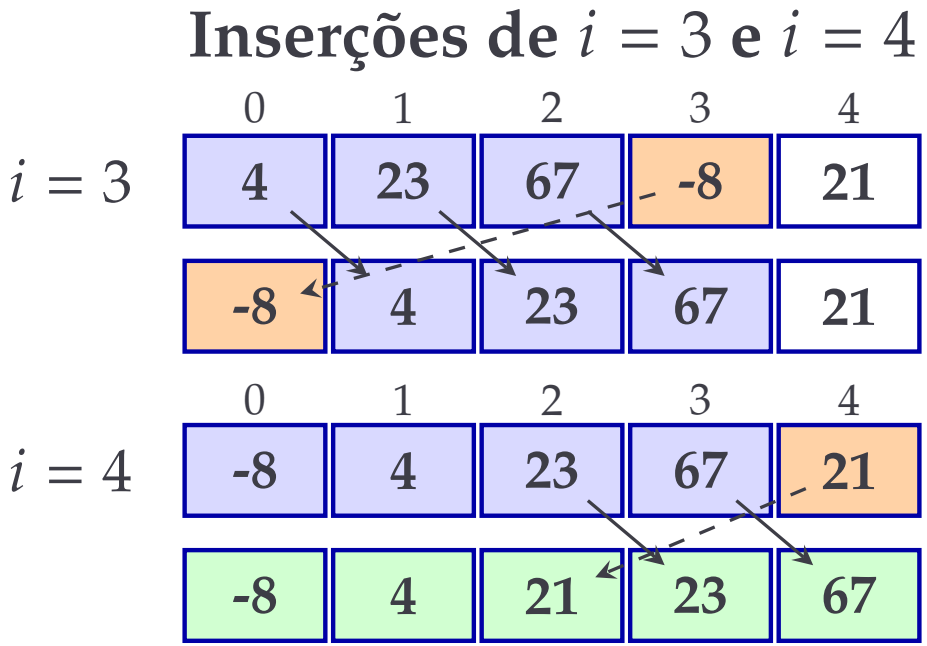
Inserções de $i = 1$ e $i = 2$



Como 23 > 4, desloca 23 e insere 4 no início.

Nenhuma chave à esquerda é maior que 67; o vetor permanece igual.

INSERTION SORT



Desloca 4, 23 e 67 para inserir -8.

Desloca 23 e 67; o vetor fica ordenado.

ATIVIDADE PRÁTICA

ATIVIDADE

Deve-se implementar os três algoritmos de ordenação

Código-base está disponível no GitHub ▸ Link

Os algoritmo pode ser visualizados em ▸ Link

PRÓXIMA AULA:

ALGORITMOS DE ORDENAÇÃO ÓTIMOS